

OpenEMMA - End to End Multimodal Large Language Model for Autonomous Driving

Poondla Latheesh (B231170EC), Hotha Kaarthikeya (B230662EC), Bhukya Jagadeesh (B230252EC),
Pranav D Pandit (B231174EC), Bommareddy Navaneet Seshi Reddy (B230261EC)

Department of Electronics and Communication Engineering
National Institute of Technology Calicut, Kerala, India

Abstract—End-to-end multimodal large language models such as Waymo’s EMMA show that a single vision-language model can map raw camera input directly to future driving trajectories. Because EMMA’s full training regime is out of reach for academic settings, OpenEMMA reproduces the idea by prompting open MLLMs with a Chain-of-Thought (CoT) procedure at inference time on nuScenes, with no fine-tuning; as part of this work we first reproduced this ourselves using LLaVA-7B and Qwen2.5-VL-3B backbones. Doing so exposed the practical limits of a text-generation-based approach: slow, autoregressive inference and fragile parsing of numeric trajectories out of free text. This project, OpenEMMA-TinyVLA, restructures the pipeline as a single trainable model, following the Vision-Language-Action (VLA) recipe introduced by TinyVLA: a compact LLaVA-Pythia backbone (1.3B parameters) adapted with LoRA at its vision encoder and language decoder, with its text-generation head replaced by a diffusion policy head that outputs a fixed-size future trajectory directly as numbers. Fine-tuned on a 150-scene slice of nuScenes and evaluated on the nuScenes-mini split, the resulting model is roughly two orders of magnitude faster at inference than the CoT baselines, though it currently trails them in trajectory accuracy; we analyze the likely causes and outline concrete next steps.

Index Terms—autonomous driving, end-to-end learning, vision-language-action models, chain-of-thought reasoning, diffusion policy, LoRA, nuScenes, trajectory prediction

I. INTRODUCTION

Autonomous driving (AD) systems have historically been built as a chain of specialized modules (perception, prediction, and planning) connected through hand-designed interfaces. This modular design is easy to build and debug component by component, but every such interface is also a place where information can be lost or wrongly discretized, and because the modules are trained separately, errors compound across module boundaries instead of being jointly optimized toward the actual goal of safe, human-like driving.

Multimodal large language models (MLLMs) offer a different path: pretrained on internet-scale data, they arrive with broad world knowledge and reasoning ability that a specialized driving stack never sees during training, and can in principle be fine-tuned end-to-end to map sensor input directly to a trajectory. Waymo’s EMMA [1] is the clearest instance of this, formulated as a single function

$$\mathbf{O}_{\text{trajectory}} = \mathcal{G}(\mathbf{T}_{\text{intent}}, \mathbf{T}_{\text{ego}}, \mathbf{V}), \quad (1)$$

with no hand-designed intermediate representation between sensing and planning. Reproducing EMMA fully is not feasi-

ble outside Waymo’s own infrastructure, since it requires large-scale proprietary driving data, a closed foundation model, and correspondingly heavy compute. OpenEMMA [2] was proposed to make the same end-to-end idea usable in an academic setting: instead of fine-tuning, it prompts an off-the-shelf, open MLLM with a Chain-of-Thought (CoT) procedure at inference time on the public nuScenes dataset, and converts the resulting speed/curvature predictions into a trajectory through a kinematic bicycle model.

As part of this project, we first reproduced OpenEMMA’s CoT-prompting pipeline ourselves, using LLaVA-7B and Qwen2.5-VL-3B as backbones, on the 10-scene nuScenes-mini split. This reproduction is what exposed OpenEMMA’s practical costs directly: because both the reasoning and the final trajectory are produced as free text, inference is slow (on the order of a minute or more per frame in our runs), and the generated text has to be parsed back into numeric speed/curvature values, a step that can fail or silently return fewer trajectory points than intended.

Our goal in this project is to keep OpenEMMA’s end-to-end aspiration (one model, raw sensor input to trajectory output, no hand-built interfaces in between) while removing exactly these costs. We do this by adopting the Vision-Language-Action (VLA) recipe introduced by TinyVLA [3] for robot manipulation: a small pretrained vision-language backbone, adapted cheaply with LoRA, whose text-generation head is replaced by a diffusion-based action decoder that outputs actions directly as numbers. Applied to driving, this yields a single unified model, OpenEMMA-TinyVLA, that takes one front-camera image, a short prompt, and the ego vehicle’s recent motion, and outputs a fixed-size future trajectory in one forward pass, with no free-text generation and no parsing step in between.

II. LITERATURE REVIEW

A. EMMA: End-to-End Multimodal Model for Autonomous Driving

EMMA [1] fine-tunes the formulation of Eq. (1) end-to-end on top of Gemini, using six-camera surround-view video, historical ego status expressed as BEV waypoints, and a router-supplied high-level intent command (e.g. “turn left”). Training is self-supervised in the sense that the only required label is the ego vehicle’s own future position. EMMA further incorporates CoT reasoning into planning by having the model articulate

a structured driving rationale before predicting waypoints, in four stages of increasing detail: a coarse scene description, a set of critical objects identified with their 3D/BEV coordinates, a behaviour description of how each critical object is expected to act, and a meta driving decision summarizing the resulting plan. These rationale captions are generated automatically, using off-the-shelf perception/prediction models plus Gemini prompting, keeping the data pipeline scalable. EMMA is trained on the Waymo Open Motion Dataset for planning (with nuScenes and the Waymo Open Dataset used for its generalist perception tasks) and reports state-of-the-art planning performance, but remains closed-source and requires data and compute far outside academic reach.

B. OpenEMMA: Open-Source Multimodal Model for End-to-End Autonomous Driving

OpenEMMA [2] treats end-to-end planning as an inference-time, prompting-only problem over off-the-shelf pretrained MLLMs (GPT-4o, LLaVA-1.6-Mistral-7B, Llama-3.2-11B-Vision-Instruct, Qwen2-VL-7B-Instruct), so no training data or compute budget beyond ordinary inference is required. Its inputs are far lighter than EMMA’s: a single front-facing camera image and the ego vehicle’s historical speed and curvature over the past five seconds, expressed as plain text.

Its CoT procedure produces three reasoning components through dedicated prompts before predicting a trajectory: a *scene description* of the driving context, the *critical object(s)* the ego vehicle needs to attend to, and an *intent description* of the ego vehicle’s own intended maneuver. The last of these stands in for the navigation-derived intent EMMA receives as an input, since nuScenes carries no such ground-truth field and the model must infer it itself. This reasoning, together with the historical ego status, is then fed back to the same MLLM to predict future *speed* and *curvature* vectors rather than raw (x, y) coordinates, which are integrated through a kinematic bicycle model: heading is obtained by integrating curvature times speed, then position is obtained by integrating the resulting velocity components, to produce the final trajectory. This choice is deliberate: no valid (speed, curvature) sequence can produce a path that teleports or reverses instantaneously, so integrating through the kinematic model guarantees a smooth, physically plausible trajectory regardless of the underlying prediction’s numerical accuracy. Before the reasoning stage, a separately fine-tuned YOLO-based detector first runs on the front-camera image, and it is this bounding-box-overlaid image, not the raw frame, that is passed into the MLLM; this is a comparatively small modification to the input rather than a fully separate post-hoc visualization step. OpenEMMA reports that CoT prompting generally reduces trajectory error and failure rate relative to a zero-shot baseline on nuScenes, but being built entirely on autoregressive text generation, it remains slow at inference and requires parsing numeric predictions back out of generated text.

C. TinyVLA: Towards Fast, Data-Efficient Vision-Language-Action Models

TinyVLA [3] addresses an analogous problem in robot manipulation: existing Vision-Language-Action (VLA) models such as OpenVLA build on large (7B+ parameter) vision-language backbones and generate discrete action tokens autoregressively, making them slow at inference and dependent on large-scale robot pretraining data. TinyVLA’s approach is two-fold. First, it initializes its policy network from a small, purpose-pretrained multimodal model rather than a large off-the-shelf one, training its own family of compact vision-language models (70M to 1.4B parameters) with Pythia as the language backbone, following LLaVA’s vision-language pretraining pipeline. Second, it discards autoregressive action-token prediction altogether and instead attaches a diffusion-based policy head directly onto the pretrained backbone, so actions are produced by a denoising process rather than by generating tokens one at a time.

During robot-data fine-tuning, TinyVLA freezes most of the pretrained backbone and adapts it with LoRA (around 5% of the transformer’s parameters are ever updated), while the diffusion policy head itself is trained with full-parameter updates. The backbone’s multimodal embeddings are pooled into a fixed-size representation, concatenated with the robot’s proprioceptive state, and used to condition a Diffusion Policy [6] head that outputs actions directly. Because inference now requires only one backbone forward pass plus a short denoising loop, rather than one forward pass per output token, TinyVLA’s largest variant (TinyVLA-H, 1.3B parameters) is reported to match or exceed OpenVLA’s manipulation success rate while using roughly 5.5 times fewer parameters and around 20 times lower inference latency, and it generalizes to novel instructions, objects and viewpoints without ever being pretrained on a large-scale robot dataset such as OpenX.

III. METHODOLOGY

A. Overview

We adapt TinyVLA’s VLA recipe to autonomous driving trajectory prediction. The backbone is LLaVA-Pythia, the 1.3B-parameter vision-language model built and pretrained by TinyVLA. We fine-tune this backbone for the driving task using Low-Rank Adaptation (LoRA) [5] applied to both the vision encoder (ViT) and the language decoder (LLM), and we replace its text-generation head entirely with a diffusion policy head (Droid Diffusion, built on a 1D conditional U-Net [6]). Because the model can no longer generate text at all, it cannot reproduce OpenEMMA’s explicit CoT text; instead, a single input prompt is used that verbally directs the model toward that same style of reasoning internally, without ever emitting it as text. Alongside the front-camera image and this prompt, the model also consumes the ego vehicle’s historical trajectory, its ten most recent (speed, curvature) observations, which is fused with the backbone’s hidden states to condition the diffusion head (Fig. 2).

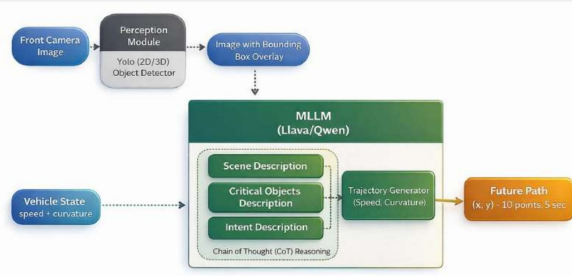


Fig. 1. OpenEMMA’s inference-time pipeline [2]: a YOLO-based detector overlays bounding boxes on the front-camera image, which together with the ego vehicle’s historic state (past speed and curvature) is passed to a CoT-prompted MLLM (LLaVA/Qwen) that produces a scene description, a critical-objects description and an intent description; a trajectory generator then converts the resulting (speed, curvature) predictions into a future path.

Fig. 1 summarizes the OpenEMMA pipeline this work starts from, for reference. A YOLO-based detector overlays bounding boxes on the front-camera image, and this overlaid image, together with the ego vehicle’s speed/curvature state, is passed to a CoT-prompted MLLM (LLaVA or Qwen), which reasons through a scene description, a critical-objects description and an intent description before a trajectory generator converts the resulting (speed, curvature) values into a 10-point, 5-second future path. OpenEMMA-TinyVLA (Fig. 2) replaces the YOLO detector and the entire CoT/trajectory-generator block with a single LoRA-adapted backbone and diffusion policy head, described next.

B. Model Architecture

Fig. 2 shows the resulting architecture, adapted from TinyVLA’s policy fine-tuning pipeline (Fig. 2 of [3]) to this task. The model takes three inputs: a single front-camera image, a short CoT-motivating prompt, and the ego vehicle’s ten most recent (speed, curvature) observations. It produces ten future (speed, curvature) pairs directly, with no free-text generation in between. The remainder of this section gives the exact formulation: Section III-C for the LoRA-adapted backbone forward pass, Section III-D for the diffusion training/inference process, and Section III-E for the conditioning and network structure of the diffusion head itself.

C. LoRA-Adapted Backbone Forward Pass

Let $I \in \mathbb{R}^{H \times W \times 3}$ be the front-camera frame and p the CoT-motivating prompt. The ViT encodes I into a sequence of patch embeddings $E_v = \text{ViT}_{\theta+\Delta\theta}(I)$, where LoRA reparameterizes every adapted linear weight $W_0 \in \mathbb{R}^{d \times k}$ inside the ViT’s attention/projection layers as

$$W = W_0 + \frac{\alpha}{r} BA, \quad B \in \mathbb{R}^{d \times r}, A \in \mathbb{R}^{r \times k}, \quad (2)$$

with rank $r = 64$ and scaling $\alpha = 256$ (so the effective update scale is $\alpha/r = 4$), B initialized to zero so training starts exactly at the pretrained solution W_0 , and only A, B receiving gradients while W_0 stays frozen. The pretrained adapter (a small MLP, frozen) then projects E_v into the

language model’s embedding space, $\tilde{E}_v = \text{Adapter}(E_v) \in \mathbb{R}^{n_v \times d_h}$, where d_h is the LLM’s hidden size. The prompt p is tokenized and embedded as $E_p \in \mathbb{R}^{n_p \times d_h}$ in the usual way, and $\tilde{E}_v$ is spliced into the token sequence in place of the image placeholder token, giving a combined sequence $E = [\dots, \tilde{E}_v, \dots, E_p, \dots] \in \mathbb{R}^{L \times d_h}$.

This sequence is processed by the Pythia (GPT-NeoX) decoder, whose linear layers (Q, K, V, O projections and MLP) are LoRA-adapted by the same reparameterization of Eq. (2), applied with independent (A, B) pairs per layer:

$$H = \text{GPTNeoX}_{\theta+\Delta\theta}(E) \in \mathbb{R}^{B \times L \times d_h}, \quad (3)$$

where H (hidden_states in code) is the decoder’s final-layer output, one d_h -dimensional vector per input position, computed under mixed precision (backbone weights in fp16, with autocast forcing LayerNorm/softmax internals to fp32 for numerical stability), then explicitly upcast to fp32 before being consumed by the diffusion head.

D. Diffusion Process Formulation

Let $a_0 \in \mathbb{R}^{10 \times 2}$ denote the ground-truth action chunk, 10 future (v, κ) pairs (chunk_size= 10, action_dim= 2). We use j for the diffusion process step throughout this section, reserving k_t for curvature as used elsewhere in this report. Training follows the DDPM/DDIM forward (noising) process with $N_{\text{train}} = 100$ diffusion steps and a squared-cosine noise schedule ($\beta_j, \bar{\alpha}_j = \prod_{i=1}^j (1 - \beta_i)$):

$$a_j = \sqrt{\bar{\alpha}_j} a_0 + \sqrt{1 - \bar{\alpha}_j} \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I), \quad (4)$$

with $j \sim \text{Uniform}\{0, \dots, N_{\text{train}} - 1\}$ sampled independently per training example. The diffusion head F_ϕ (embed_out, trained with full-parameter updates) is trained to predict the injected noise itself (prediction_type=’epsilon’) rather than the clean action:

$$\hat{\varepsilon} = F_\phi(a_j, j, c), \quad c = c(H, s), \quad (5)$$

where $c(H, s)$ is the conditioning vector built from the backbone output H and the historical ego-state vector $s \in \mathbb{R}^{20}$ (defined in Eq. (9) below), and the training objective is a padding-masked mean-squared error over the predicted noise,

$$\mathcal{L} = \mathbb{E}_{a_0, \varepsilon, j} \left[\|\hat{\varepsilon} - \varepsilon\|_2^2 \odot (1 - m) \right], \quad (6)$$

with m a binary padding mask ($m_t = 1$ for padded future steps) so that padded timesteps do not contribute to the loss.

At inference, no ground-truth action exists, so the action chunk is initialized from pure Gaussian noise and denoised deterministically ($\eta = 0$ DDIM) over $N_{\text{infer}} = 10$ re-spaced steps $j_N > j_{N-1} > \dots > j_1$ (a subsequence of the 100 training steps):

$$a_{j_N} \sim \mathcal{N}(0, I), \quad \hat{a}_0^{(j)} = \frac{a_j - \sqrt{1 - \bar{\alpha}_j} \hat{\varepsilon}_j}{\sqrt{\bar{\alpha}_j}}, \quad (7)$$

$$a_{j^-} = \sqrt{\bar{\alpha}_{j^-}} \hat{a}_0^{(j)} + \sqrt{1 - \bar{\alpha}_{j^-}} \hat{\varepsilon}_j, \quad (8)$$

where $\hat{\varepsilon}_j = F_\phi(a_j, j, c)$ and j^- is the next (smaller) step in the re-spaced schedule. After the final step, $a_0 \in \mathbb{R}^{10 \times 2}$ is

returned directly as the predicted future (v, κ) sequence: one backbone forward pass (Eq. (3)) plus $N_{\text{infer}} = 10$ evaluations of F_ϕ , versus one backbone forward pass per generated token for OpenEMMA’s autoregressive text output.

E. Diffusion Head Network Architecture

The conditioning vector c in Eq. (5) is built in two stages. First, the backbone output $H \in \mathbb{R}^{B \times L \times d_h}$ is reduced to a fixed-size, sequence-length-independent vector by average pooling over the token axis, layer-normalized, concatenated with the ego-state vector s , and linearly recombined:

$$c_H = \text{Linear}_{d_h+20 \rightarrow d_h} \left(\left[\text{LayerNorm}(\text{AvgPool}_L(H)); s \right] \right) \in \mathbb{R}^{d_h}. \quad (9)$$

Second, the diffusion step j is embedded with a sinusoidal positional encoding followed by a 2-layer MLP,

$$\tau(j) = W_2 \text{Mish}(W_1 \text{SinEmb}_{256}(j)) \in \mathbb{R}^{256}, \quad (10)$$

and the two are concatenated into the full conditioning signal used throughout the network, $c = [\tau(j); c_H] \in \mathbb{R}^{256+d_h}$.

F_ϕ itself is a 1D convolutional U-Net (ConditionalUnet1D) operating over the 10-step action sequence (channel dimension = 2), with encoder channel widths [256, 512, 1024], kernel size 5 and 8-group GroupNorm. Every residual block applies FiLM conditioning¹: given a hidden feature map h with C_{out} channels, c is projected to a per-channel scale and shift,

$$[\gamma; \beta] = \text{Linear}_{(256+d_h) \rightarrow 2C_{\text{out}}}(\text{Mish}(c)), \quad \gamma, \beta \in \mathbb{R}^{C_{\text{out}}}, \quad (11)$$

$$\text{FiLM}(h, c) = \gamma \odot h + \beta, \quad (12)$$

broadcast over the time axis, followed by a second convolutional block and a residual connection (a 1×1 convolution when the channel count changes). The network has two downsampling stages (stride-2 causal convolutions, $10 \rightarrow 256\text{ch} \rightarrow 512\text{ch}$), a two-block bottleneck at the widest level (1024 channels), and two matching upsampling stages (transposed convolutions) with U-Net-style skip connections from the encoder; a final 1×1 convolution projects back down to the 2-channel (v, κ) output. Because two real downsampling stages only round-trip exactly when the horizon length is a multiple of $2^2 = 4$, the 10-step action chunk is zero-padded to 12 internally and cropped back to 10 after the final convolution.

F. Kinematic Bicycle-Model Integration

The diffusion head’s raw output is a sequence of future (speed, curvature) pairs, $\{(s_t, k_t)\}_{t=1}^{10}$, not (x, y) coordinates directly. To obtain a trajectory for evaluation and visualization,

¹FiLM: Perez et al., “FiLM: Visual Reasoning with a General Conditioning Layer,” AAAI 2018.

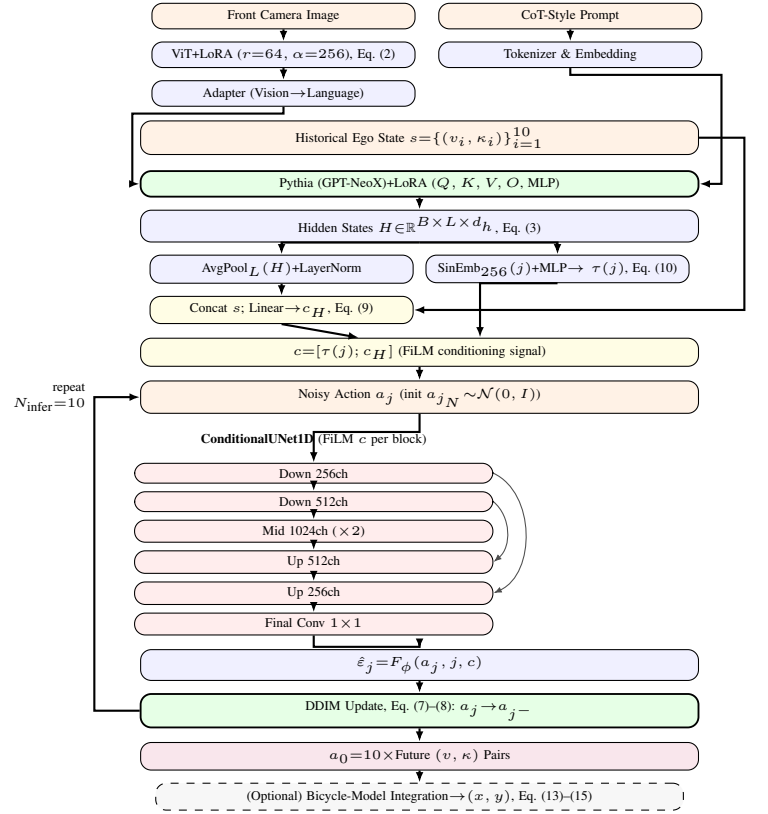


Fig. 2. OpenEMMA-TinyVLA architecture. A LoRA-adapted vision-language backbone (LLaVA-Pythia) jointly encodes the front-camera image and CoT-style prompt into hidden states H ; these are pooled, normalized and combined with the ego vehicle’s historical state into c_H , which is concatenated with a sinusoidal diffusion-step embedding $\tau(j)$ to form the FiLM conditioning signal c . A 1D convolutional U-Net (ConditionalUnet1D) denoises an initially random action chunk a_{jN} over $N_{\text{infer}} = 10$ DDIM steps, conditioned by c at every residual block, directly into ten future (speed, curvature) pairs. No free-text generation and no external detection module are used in the prediction path.

we integrate this sequence through the same kinematic bicycle-model relations used by OpenEMMA. The heading θ_t is obtained by integrating curvature times speed,

$$\theta_t = \theta_{t-1} + \sum_{i=1}^t k_i s_i \Delta t, \quad (13)$$

after which the velocity components in the x and y directions follow as

$$v_x(t) = s_t \cos \theta_t, \quad v_y(t) = s_t \sin \theta_t, \quad (14)$$

and the resulting position is obtained by integrating these velocity components,

$$x_t = x_{t-1} + \sum_{i=1}^t v_x(i) \Delta t, \quad y_t = y_{t-1} + \sum_{i=1}^t v_y(i) \Delta t, \quad (15)$$

with $\Delta t = 0.5\text{s}$, the interval between consecutive nuScenes keyframes (Section III-H), and (x_0, y_0, θ_0) the ego vehicle’s pose at the current frame. Predicting (speed, curvature) and integrating through Eqs. (13)–(15), rather than regressing

(x, y) directly, guarantees that the resulting trajectory is always smooth and kinematically consistent: no (speed, curvature) sequence can produce a path that teleports or reverses instantaneously, whatever the numerical accuracy of the underlying prediction.

G. Diffusion Head vs. Autoregressive Text Generation

Replacing text generation with a diffusion head changes the output mechanism in three ways that matter in practice. First, the diffusion head produces a *fixed-size* numeric output (ten (speed, curvature) pairs) by construction, whereas OpenEMMA’s text output is variable-length and can yield fewer than the requested number of future points, or values that fail to parse, when the model’s generated text does not follow the expected pattern. Second, autoregressive generation is a loop bounded by a maximum token count, requiring one backbone forward pass per generated token, across several sequential CoT prompts; the diffusion head instead requires one backbone forward pass followed by a short, fixed number of denoising steps, regardless of how much text a CoT explanation would otherwise have taken (Section IV-C reports the resulting difference in measured inference time). Third, removing text generation removes the parsing step entirely: there is no regex extraction of numbers from a sentence, and therefore no failure mode where that extraction goes wrong.

H. Dataset and Input Construction

Both fine-tuning and evaluation use nuScenes [4], a large-scale driving dataset of 1,000 twenty-second scenes recorded in Boston and Singapore with six cameras, one lidar and five radars, annotated with 3D object boxes and HD maps. Of this, we use only two things: the front-camera RGB image, and each sample’s `ego_pose` record, from which we derive the ego vehicle’s speed and curvature history; we do not use lidar, radar, the HD maps, or the 3D box annotations. nuScenes releases fully annotated *keyframes* at 2 frames per second; we sample our historical and future windows at this native keyframe rate, so the 10 historical (speed, curvature) observations span the preceding 5 seconds and the 10 predicted future pairs span the following 5 seconds, the same horizon OpenEMMA itself predicts over. Fine-tuning uses a 150-scene slice of nuScenes, drawing supervision only from each sample’s `ego_pose` so that no withheld annotation labels are required; evaluation uses the 10-scene nuScenes-mini split.

I. Training Setup

The backbone is adapted with LoRA (rank 64, $\alpha = 256$, dropout 0.05) applied to the linear layers of both the ViT and the LLM; the diffusion policy head’s parameters are trained in full. Fine-tuning runs for 5 epochs, with a batch size of 2 and gradient accumulation, using two separate learning rates for the LoRA-adapted backbone and the diffusion head (dataset and split details are as described in Section III-H).

TABLE I
OPENEMMA-TINYVLA PER-SCENE ADE (M) ON NUSCENES-MINI (10 SCENES)

Scene	ADE@1s	ADE@2s	ADE@3s	avg ADE
scene-0061	2.955	4.851	6.693	4.833
scene-0103	2.771	4.641	6.591	4.668
scene-0553	0.339	0.480	0.604	0.474
scene-0655	5.507	9.208	12.946	9.220
scene-0757	0.640	1.025	1.349	1.005
scene-0796	8.478	14.170	19.942	14.196
scene-0916	3.270	5.541	7.815	5.542
scene-1077	9.551	16.119	22.810	16.160
scene-1094	4.215	7.451	11.001	7.556
scene-1100	0.359	0.582	0.785	0.575
Mean	3.809	6.407	9.054	6.423

TABLE II
EARLIER OPENEMMA (CoT-PROMPTING, NO FINE-TUNING)
REPRODUCTION ON NUSCENES-MINI (10 SCENES)

Model	ADE@1s	ADE@2s	ADE@3s	avg ADE
LLaVA-7B	1.90	1.82	2.50	2.07
Qwen2.5-VL-3B	0.87	1.52	2.19	1.53

IV. RESULTS

A. Evaluation Protocol

We report Average Displacement Error (ADE, in meters) between the predicted and ground-truth ego trajectory at 1, 2 and 3 seconds into the future, together with their per-scene average, on the 10 scenes of nuScenes-mini. For a horizon of T keyframe steps, ADE is the mean ℓ_2 distance between predicted and ground-truth position over that horizon,

$$\text{ADE}_T = \frac{1}{T} \sum_{t=1}^T \|(\hat{x}_t, \hat{y}_t) - (x_t, y_t)\|_2, \quad (16)$$

where $(\hat{x}_t, \hat{y}_t)$ is the trajectory predicted via Eqs. (13)–(15) and (x_t, y_t) is the corresponding ground-truth ego position at step t ; at the 2 fps keyframe rate, $T = 2, 4, 6$ steps correspond to the 1, 2 and 3 second horizons, respectively. The avg ADE column in Table I is the mean of ADE_2 , ADE_4 and ADE_6 for that scene. A scene is additionally marked as a failure when the model’s rollout becomes numerically unstable during generation.

B. Quantitative Results

Table I reports per-scene results for the current OpenEMMA-TinyVLA checkpoint (LoRA + diffusion head, 5 epochs).

The mean ADE across all 10 scenes is 6.42 m, with one scene (scene-1077) additionally showing a 33.3% failure rate during rollout.

For context, Table II reports our earlier reproduction of OpenEMMA’s own CoT-prompting pipeline (no fine-tuning), on the same nuScenes-mini split.

Both baselines score well below OpenEMMA-TinyVLA’s current 6.42 m mean ADE (Table I), and both are dominated by

a handful of simple, near-stationary or straight-line scenes such as scene-0553, which is also where OpenEMMA-TinyVLA performs closest to them (0.47 m).

C. Inference Speed

The motivation for this redesign (Section III-G) is most visible in measured inference latency. The CoT-prompting baselines, which require several sequential rounds of autoregressive text generation, run at approximately 90 seconds per frame. OpenEMMA-TinyVLA runs at approximately 1 second per frame on average, roughly two orders of magnitude faster, and well within the range needed for the model to eventually be considered for anything resembling real-time use.

D. Discussion: Why Accuracy Currently Trails the CoT Baselines

Despite the large speed advantage, OpenEMMA-TinyVLA’s trajectory accuracy currently trails both CoT baselines by a wide margin. We attribute this primarily to insufficient and imbalanced fine-tuning rather than to a fundamental limitation of the approach:

- **Training length.** The model was fine-tuned for only 5 epochs, which is likely too few for a diffusion-based action head to converge, especially compared to LoRA’s fast adaptation of an already-pretrained backbone.
- **A from-scratch, harder-to-train component.** The diffusion policy head has no prior exposure to driving data and is effectively being trained from a much less favorable starting point than the LoRA-adapted backbone, which only has to nudge an already strong pretrained representation. This creates a training trade-off between a component that adapts easily (LoRA) and one that must learn its entire function from limited supervision (the diffusion head).
- **Small training data.** Fine-tuning was performed on a 150-scene slice of nuScenes, which is small relative to the diversity of real driving scenarios the model needs to generalize across.

Beyond the training shortfall, there is also a missing input: nuScenes provides no ground-truth navigation intent, so unlike EMMA, which receives a router-supplied intent command directly, OpenEMMA at least has its CoT text infer and state an intent explicitly before predicting a trajectory, whereas OpenEMMA-TinyVLA has no explicit representation of intent at all. This may compound its error specifically at genuinely multi-modal decision points (e.g. junctions), where the correct trajectory depends on an intention that cannot be recovered from the image and past motion alone.

Based on this, the most promising directions for closing the accuracy gap are: training for substantially more epochs; fine-tuning on the full nuScenes dataset rather than a 150-scene slice, to give the diffusion head enough varied supervision; and revisiting the learning-rate balance between the two trained components: in particular, using a comparatively slower learning rate for the already-pretrained LoRA parameters and a comparatively higher learning rate for the diffusion head, since

the two components are starting from very different points and are unlikely to need the same training dynamics.

REFERENCES

- [1] J.-J. Hwang, R. Xu, H. Lin, W.-C. Hung, J. Ji, K. Choi, D. Huang, T. He, P. Covington, B. Sapp, Y. Zhou, J. Guo, D. Anguelov, and M. Tan, “EMMA: End-to-End Multimodal Model for Autonomous Driving,” *Transactions on Machine Learning Research*, 2025. [Online]. Available: arXiv:2410.23262.
- [2] S. Xing, C. Qian, Y. Wang, H. Hua, K. Tian, Y. Zhou, and Z. Tu, “OpenEMMA: Open-Source Multimodal Model for End-to-End Autonomous Driving,” arXiv preprint arXiv:2412.15208, 2024.
- [3] J. Wen, Y. Zhu, J. Li, M. Zhu, Z. Tang, K. Wu, Z. Xu, N. Liu, R. Cheng, C. Shen, Y. Peng, F. Feng, and J. Tang, “TinyVLA: Towards Fast, Data-Efficient Vision-Language-Action Models for Robotic Manipulation,” *IEEE Robotics and Automation Letters*, 2025. [Online]. Available: arXiv:2409.12514.
- [4] H. Caesar, V. Bankiti, A. H. Lang, S. Vora, V. E. Liong, Q. Xu, A. Krishnan, Y. Pan, G. Baldan, and O. Beijbom, “nuScenes: A Multimodal Dataset for Autonomous Driving,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2020.
- [5] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-Rank Adaptation of Large Language Models,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2022.
- [6] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, and S. Song, “Diffusion Policy: Visuomotor Policy Learning via Action Diffusion,” in *Proc. Robotics: Science and Systems (RSS)*, 2023.
- [7] H. Liu, C. Li, Q. Wu, and Y. J. Lee, “Visual Instruction Tuning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [8] S. Biderman, H. Schoelkopf, Q. G. Anthony, H. Bradley, K. O’Brien, E. Hallahan, M. A. Khan, S. Purohit, U. S. Prashanth, E. Raff, A. Skowron, L. Sutawika, and O. van der Wal, “Pythia: A Suite for Analyzing Large Language Models Across Training and Scaling,” in *Proc. Int. Conf. Machine Learning (ICML)*, 2023.
- [9] J. Ho, A. Jain, and P. Abbeel, “Denoising Diffusion Probabilistic Models,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.